7주차 / BSTTest.java

알고리즘 · 실습 코드

```
class TreeNode {
    String key;
    TreeNode Lchild;
    TreeNode Rchild;
}
class BinarySearchTree {
    private TreeNode rootNode;
    public TreeNode BSTsearch(String K) {
        TreeNode p = rootNode;
        while (p != null) {
            int result = K.compareTo(p.key);
            if (result == 0)
            return p;
            else if (result < 0)
            p = p.Lchild;
            else
            p = p.Rchild;
        }
        return null;
    }
    public void BSTinsert(String K) {
        TreeNode newNode = new TreeNode();
        newNode.key = K;
        newNode.Lchild = null;
        newNode.Rchild = null;
        if (rootNode == null) {
            rootNode = newNode;
            return;
        }
        TreeNode p = rootNode;
        TreeNode parent = null;
        while (p != null) {
            parent = p;
            int result = K.compareTo(p.key);
            if (result == 0)
            return;
            else if (result < 0)
            p = p.Lchild;
            else
            p = p.Rchild;
        }
        if (K.compareTo(parent.key) < 0)
        parent.Lchild = newNode;
        else
        parent.Rchild = newNode;
    }
    private TreeNode maxNode(TreeNode root) {
```

```
48          if (root == null)
49          return null;
50          while (root.Rchild != null) {
51              root = root.Rchild;
52          }
53          return root;
54      }
55      private TreeNode delete(TreeNode root, String K) {
56          if (root == null)
57          return null;
58          int result = K.compareTo(root.key);
59          if (result < 0) {
60              root.Lchild = delete(root.Lchild, K);
61          } else if (result > 0) {
62              root.Rchild = delete(root.Rchild, K);
63          } else {
64              // 자식이 없는 경우 또는 오른쪽 자식만 있는 경우
65              if (root.Lchild == null)
66              return root.Rchild;
67              // 왼쪽 자식만 있는 경우
68              if (root.Rchild == null)
69              return root.Lchild;
70              // 자식이 두 개 있는 경우
71              TreeNode temp = maxNode(root.Lchild);
72              root.key = temp.key;
73              root.Lchild = delete(root.Lchild, temp.key);
74          }
75          return root;
76      }
77      public void BSTdelete(String K) {
78          rootNode = delete(rootNode, K);
79      }
80      public boolean BSTsplit(String x, BinarySearchTree SmallTree, BinarySearchTr
 > ee LargeTree) {
81          TreeNode Small = new TreeNode();
82          TreeNode Large = new TreeNode();
83          TreeNode S = Small;
84          TreeNode L = Large;
85          TreeNode P = rootNode;
86          while (P != null) {
87              int result = x.compareTo(P.key);
88              if (result == 0) {
89                  S.Rchild = P.Lchild;
90                  L.Lchild = P.Rchild;
91                  SmallTree.rootNode = Small.Rchild;
92                  LargeTree.rootNode = Large.Lchild;
93                  return true;
```

```
            } else if (result < 0) {
                L.Lchild = P;
                L = P;
                P = P.Lchild;
            } else {
                S.Rchild = P;
                S = P;
                P = P.Rchild;
            }
        }
        S.Rchild = null;
        L.Lchild = null;
        SmallTree.rootNode = Small.Rchild;
        LargeTree.rootNode = Large.Lchild;
        return false;
    }
    private void printNode(TreeNode n) {
        if (n != null) {
            System.out.print("(");
            printNode(n.Lchild);
            System.out.print(" " + n.key + " ");
            printNode(n.Rchild);
            System.out.print(")");
        }
    }
    public void print() {
        printNode(rootNode);
        System.out.println();
    }
}
public class BSTTest {
    public static void main(String args[]) {
        BinarySearchTree t = new BinarySearchTree();
        t.BSTinsert("S");
        t.BSTinsert("J");
        t.BSTinsert("B");
        t.BSTinsert("D");
        t.BSTinsert("U");
        t.BSTinsert("M");
        t.BSTinsert("R");
        t.BSTinsert("Q");
        t.BSTinsert("A");
        t.BSTinsert("G");
        t.BSTinsert("E");
        System.out.println("===== 초기 이진 탐색 트리 =====");
        t.print();
        System.out.println("===== R 검색 =====");
```

```
141         TreeNode n = t.BSTsearch("R");
142         if (n == null)
143         System.out.println("Not Found");
144         else
145         System.out.println("Found");
146         System.out.println("===== R 삭제 후 검색 =====");
147         t.BSTdelete("R");
148         n = t.BSTsearch("R");
149         if (n == null)
150         System.out.println("Not Found");
151         else
152         System.out.println("Found");
153         System.out.println("R 삭제 후 트리");
154         t.print();
155         System.out.println("===== E 검색 =====");
156         n = t.BSTsearch("E");
157         if (n == null)
158         System.out.println("Not Found");
159         else
160         System.out.println("Found");
161         System.out.println("===== E 삭제 후 검색 =====");
162         t.BSTdelete("E");
163         n = t.BSTsearch("E");
164         if (n == null)
165         System.out.println("Not Found");
166         else
167         System.out.println("Found");
168         System.out.println("E 삭제 후 트리");
169         t.print();
170         System.out.println("===== J 검색 및 삭제 =====");
171         n = t.BSTsearch("J");
172         if (n == null)
173         System.out.println("Not Found");
174         else
175         System.out.println("Found");
176         t.BSTdelete("J");
177         System.out.println("J 삭제 후 검색");
178         n = t.BSTsearch("J");
179         if (n == null)
180         System.out.println("Not Found");
181         else
182         System.out.println("Found");
183         System.out.println("J 삭제 후 트리");
184         t.print();
185         System.out.println("===== S 검색 및 삭제 =====");
186         n = t.BSTsearch("S");
187         if (n == null)
```

```
188         System.out.println("Not Found");
189         else
190         System.out.println("Found");
191         t.BSTdelete("S");
192         System.out.println("S 삭제 후 검색");
193         n = t.BSTsearch("S");
194         if (n == null)
195         System.out.println("Not Found");
196         else
197         System.out.println("Found");
198         System.out.println("S 삭제 후 트리");
199         t.print();
200         System.out.println("===== B 검색 및 삭제 =====");
201         n = t.BSTsearch("B");
202         if (n == null)
203         System.out.println("Not Found");
204         else
205         System.out.println("Found");
206         t.BSTdelete("B");
207         System.out.println("B 삭제 후 검색");
208         n = t.BSTsearch("B");
209         if (n == null)
210         System.out.println("Not Found");
211         else
212         System.out.println("Found");
213         System.out.println("B 삭제 후 트리");
214         t.print();
215         System.out.println("===== 존재하지 않는 C 검색 =====");
216         n = t.BSTsearch("C");
217         if (n == null)
218         System.out.println("Not Found");
219         else
220         System.out.println("Found");
221         System.out.println("===== 새로운 이진 탐색 트리 t1 생성 =====");
222         BinarySearchTree t1 = new BinarySearchTree();
223         t1.BSTinsert("S");
224         t1.BSTinsert("J");
225         t1.BSTinsert("B");
226         t1.BSTinsert("D");
227         t1.BSTinsert("U");
228         t1.BSTinsert("M");
229         t1.BSTinsert("R");
230         t1.BSTinsert("Q");
231         t1.BSTinsert("A");
232         t1.BSTinsert("G");
233         t1.BSTinsert("E");
234         System.out.println("t1 트리");
```

```
235         t1.print();
236         System.out.println("===== t1에서 R 검색 =====");
237         n = t1.BSTsearch("R");
238         if (n == null)
239         System.out.println("Not Found");
240         else
241         System.out.println("Found");
242         System.out.println("===== t1에서 C 검색 =====");
243         n = t1.BSTsearch("C");
244         if (n == null)
245         System.out.println("Not Found");
246         else
247         System.out.println("Found");
248         BinarySearchTree Small = new BinarySearchTree();
249         BinarySearchTree Large = new BinarySearchTree();
250         System.out.println("===== t 트리를 D를 기준으로 분할 =====");
251         t.BSTsplit("D", Small, Large);
252         System.out.println("D보다 작은 트리");
253         Small.print();
254         System.out.println("D보다 큰 트리");
255         Large.print();
256         System.out.println("===== t1 트리를 C를 기준으로 분할 =====");
257         t1.BSTsplit("C", Small, Large);
258         System.out.println("C보다 작은 트리");
259         Small.print();
260         System.out.println("C보다 큰 트리");
261         Large.print();
262         System.out.println("===== T를 루트로 하는 새로운 트리 생성 =====");
263         t1 = new BinarySearchTree();
264         t1.BSTinsert("T");
265         t1.BSTinsert("U");
266         t1.BSTinsert("S");
267         t1.BSTinsert("R");
268         t1.BSTinsert("O");
269         t1.BSTinsert("M");
270         t1.BSTinsert("W");
271         t1.BSTinsert("X");
272         t1.BSTinsert("C");
273         t1.BSTinsert("B");
274         t1.BSTinsert("A");
275         System.out.println("T 삭제 전 트리");
276         t1.print();
277         System.out.println("===== T 삭제 =====");
278         t1.BSTdelete("T");
279         System.out.println("T 삭제 후 트리");
280         t1.print();
281         System.out.println("===== X, R 삭제 테스트를 위한 새로운 트리 생성 =====");
```

```
282             t1 = new BinarySearchTree();
283             t1.BSTinsert("T");
284             t1.BSTinsert("U");
285             t1.BSTinsert("R");
286             t1.BSTinsert("S");
287             t1.BSTinsert("O");
288             t1.BSTinsert("M");
289             t1.BSTinsert("X");
290             t1.BSTinsert("W");
291             t1.BSTinsert("C");
292             t1.BSTinsert("B");
293             t1.BSTinsert("Y");
294             System.out.println("삭제 전 트리");
295             t1.print();
296             System.out.println("===== X 삭제 =====");
297             t1.BSTdelete("X");
298             System.out.println("X 삭제 후 트리");
299             t1.print();
300             System.out.println("===== R 삭제 =====");
301             t1.BSTdelete("R");
302             System.out.println("R 삭제 후 트리");
303             t1.print();
304         }
305 }
```